

Distributed Systems

1 Introduction

In a **distributed database system**, data is stored across multiple nodes located at **geographically separated sites**. These nodes communicate via various media such as high-speed private networks or the internet, but they do **not share main memory or disks**.

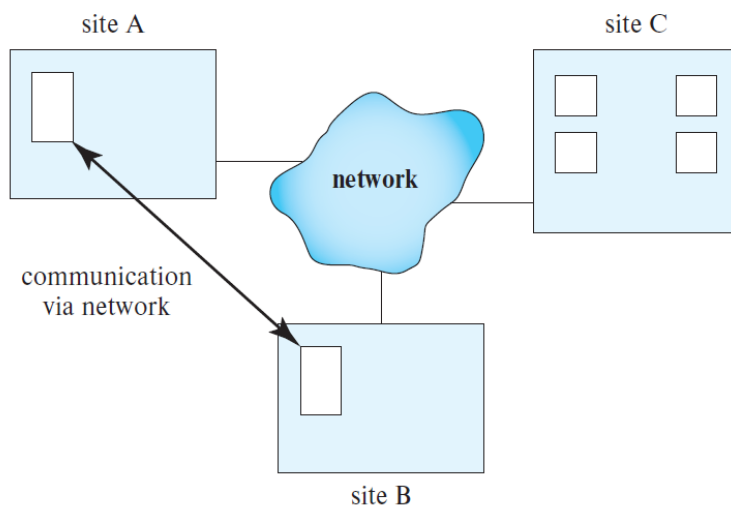


Figure 1: General structure of a distributed system.

2 Parallel vs. Distributed Databases

While both parallel and distributed databases use multiple nodes, there are fundamental differences:

Aspect	Distributed Systems
Geographic Separation	Sites are geographically separated, resulting in higher latency, lower bandwidth, and increased failure probability.
Disaster Recovery	Must handle entire data center failures (earthquakes, fires). Requires replication across geographically separated data centers.
Administration	Often separately administered with each site retaining autonomy. May result from integrating existing databases.
Node Heterogeneity	Nodes vary more in size and function compared to parallel systems.

3 Transaction Types

Local vs. Global Transactions

- **Local Transaction:** Accesses data only from the node where it was initiated.
- **Global Transaction:** Accesses data from different nodes or multiple nodes.

4 Web-Scale Systems

Modern web-scale applications combine parallelism and distribution:

- **Parallelism within data centers:** Handles high loads.
- **Distribution across data centers:** Ensures high availability during natural disasters.

Functionality Spectrum

Systems range from simple distributed key-value stores (limited functionality) to full distributed databases with schemas, query languages, and transactions. Web-scale systems are typically **centrally administered**.

5 Integrated Distributed Databases

Systems built by integrating existing databases have unique characteristics:

5.1 Data Sharing

Users at one site can access data at other sites. For example, in a distributed university system, student records can be accessed across campuses without external data transfer mechanisms.

5.2 Autonomy

Each site retains control over locally stored data. A **global database administrator** manages the entire system, but delegates responsibilities to **local administrators** at each site.

5.3 Homogeneous vs. Heterogeneous Systems

- **Homogeneous:** Nodes share a common global schema, run the same DBMS software, and actively cooperate in processing.
- **Heterogeneous (Federated):** Multiple existing systems with different schemas and DBMS software. Sites may have limited awareness of each other and provide minimal cooperation.

6 Wide-Area Network Challenges

Distributed databases communicate over **Wide-Area Networks (WAN)**, which present unique challenges:

6.1 Bandwidth Limitations

WAN bandwidth is shared among multiple users/applications and is expensive relative to LAN bandwidth, resulting in lower effective bandwidth for individual applications.

6.2 Latency

Fundamental Latency Problem

Messages may take up to **hundreds of milliseconds** to traverse the globe due to:

- **Speed-of-light delays:** Physical limitation that cannot be reduced.
- **Queuing delays:** At routers along the message path.

Applications must be carefully designed to minimize latency impact on performance.

6.3 Network Partitions

WAN link failures can result in **network partitions**, where two alive sites cannot communicate. This affects system **availability** as users may not be able to access required data.

Availability vs. Consistency

Network partitions create tradeoffs between data availability and consistency. These tradeoffs are critical in distributed system design.

7 System Availability

7.1 Single Point of Failure

If all nodes are required for the system to function, failure of even a single node can stop the entire system from operating.

7.2 Availability Through Redundancy

Higher system availability can be achieved through **data replication**:

- Data can be replicated at remote sites.
- The system can continue functioning even if one or more sites fail.
- Provides fault tolerance against both node failures and network partitions.

8 Implementation Issues

Distributed databases face unique implementation challenges that require specialized protocols and mechanisms.

8.1 Atomicity in Distributed Transactions

Two-Phase Commit Protocol (2PC)

Ensures atomicity for transactions that update data at multiple sites:

1. **Phase 1 (Prepare):** Each site executes the transaction until just before commit, then waits for the coordinator's decision.
2. **Phase 2 (Commit/Abort):** The coordinator makes the final decision, and all sites must follow it, even if there are failures while waiting.

Alternative Transaction Models

2PC is not always appropriate. Other models include:

- **Persistent Messaging:** Asynchronous message-based coordination.
- **Workflows:** Long-running transactions with compensation mechanisms.

8.2 Distributed Concurrency Control

Concurrency control mechanisms must coordinate across multiple sites to prevent conflicts and ensure serializability. This includes:

- **Distributed Locking:** Coordinating locks across sites.
- **Distributed Deadlock Detection:** Identifying and resolving deadlocks that span multiple nodes.

8.3 Data Replication

Data items may be replicated to improve availability, but this introduces challenges:

- **Consistency Maintenance:** Ensuring all replicas remain synchronized.
- **Update Propagation:** Distributing changes to all replica sites.
- **Replica Selection:** Choosing which replica to read from for optimal performance.